4.3. Longest Common Subsequence

"The shortest path between two strings."

Main Section:

4. Dynamic Programming

Contents:

At first glance, the idea of a "Longest Common Subsequence" sounds deceptively simple. Two strings are given, and somehow we are supposed to find the longest sequence that exists in both of them. Most beginners immediately think: ***"If the same characters appear in both strings... then that must be the answer."***

And honestly? That instinct is not entirely wrong. But it is incomplete. Because the real challenge is not about whether characters merely exist in both strings. The challenge is whether those characters preserve their order.

Imagine two strings:

$$\text{S1} = \text{“ABCDEFGHI”}$$

$$\text{S2} = \text{“CDGI”}$$

Here, the answer seems obvious. Every character in S2 appears in S1, and more importantly, they appear in the exact same order. So the longest common subsequence becomes: “CDGI”

Simple enough. But now let's slightly alter the second string:

$$\text{S1} = \text{“ABCDEFGHI”}$$

$$\text{S2} = \text{“ECDGI”}$$

Now things become interesting. At first, you might still think the answer is “ECDGI” because every character exists in both strings. But the moment we examine the order carefully, the illusion breaks.

- Inside S1, the character "E" appears after "C".
- But in S2, "E" comes before "C".

That means the sequence no longer preserves the same ordering between the two strings.

And this is the core idea behind subsequences. A subsequence is formed by removing some characters from a string without changing the order of the remaining characters. The characters do not need to stay adjacent, but they must maintain their relative positioning.

This is what makes a subsequence different from a substring. A substring requires the characters to remain consecutive. A subsequence only requires them to remain ordered.

- So the problem is no longer: "Which characters are common?"
- Instead, the real question becomes: "What is the longest ordered sequence that both strings can still agree on?"

And surprisingly, that tiny difference turns the problem into one of the most important Dynamic Programming problems in Computer Science.

1. Problem Definition

Now let us properly define the problem. Given two strings, the goal is to determine the longest sequence of characters that appears in both strings while preserving the same order.

Notice something important here: The characters do not need to be consecutive. They simply need to appear in the same relative order. Let us observe a smaller example:

$$\text{S1} = \text{“ABCD”}$$

$$\text{S2} = \text{“CAD”}$$

Suppose we analyze S2 while trying to build a subsequence inside S1.

- The first character in S2 is: "C". Inside S1, "C" is found in the third position. So we include it.
- Next comes: "A". "A" exists in S1, but it appears before "C". That violates the ordering requirement.
- Once we commit to "C", every future character we choose must appear after "C" in S1. So "A" is rejected.
- Finally "D", since it appears after "C" in S1, so it is included. This forms the subsequence: "CD"

And this small example reveals the true nature of the problem. The algorithm is constantly making decisions: Include this character? Skip this character? Does this preserve ordering? Will this lead to a longer subsequence later?

Now let us return to the earlier example:

$$\text{S1} = \text{“ABCDEFGHI”}$$

$$\text{S2} = \text{“ECDGI”}$$

If we carefully analyze all valid ordered subsequences, we may obtain sequences such as: "CD"; "CDI"; "CGI"; "EGI"; "$CDGI$". Among all of them, the longest valid subsequence becomes: "$CDGI$"

And this is why the problem is more complicated than simply finding repeated characters. The algorithm must simultaneously track character matches, preserve ordering, explore different possibilities, compare competing subsequences

In other words, the problem is fundamentally about structured decision-making. And because many smaller decisions repeat themselves again and again, this naturally becomes a Dynamic Programming problem.

The "Longest Common Subsequence" is therefore:

> ***The longest ordered sequence of characters that appears in both strings, not necessarily contiguously.***

And although common substrings are often also common subsequences, the reverse is not always true. A subsequence cares about order. A substring cares about adjacency. That single distinction changes everything.

2. Solution using Backtracking

Now let's finally solve the problem. Interestingly, the Longest Common Subsequence problem can be solved using two major approaches: Backtracking and Dynamic Programming

And honestly, the Backtracking solution is exactly what you would expect from a human trying to manually explore every possible sequence.

The idea is straightforward at first. We begin from the first character of both strings and compare them one by one. Suppose we are currently examining two characters: One from S1 and One from S2. Now two situations may occur.

- If the characters match, then things become simple. That character can safely become part of the subsequence. So we include it and continue exploring the next character in both strings simultaneously.
- But if the characters do not match, that's where the real branching begins. Because now we face a decision. Which character should we ignore?

And Backtracking answers this question in the most brute-force way possible: ***"Try both".***

So whenever the characters differ, we split the problem into two recursive possibilities:

- Skip the current character in the first string while keeping the current character in the second string.
- Skip the current character in the second string while keeping the current character in the first string.

And then, we recursively continue exploring both possibilities. This creates a massive decision tree. Every mismatch causes branching. Every branch explores another possible subsequence. Every recursive path attempts to build a valid ordered sequence.

Eventually, some paths become invalid, some produce shorter subsequences, and some surprisingly lead to the optimal solution.

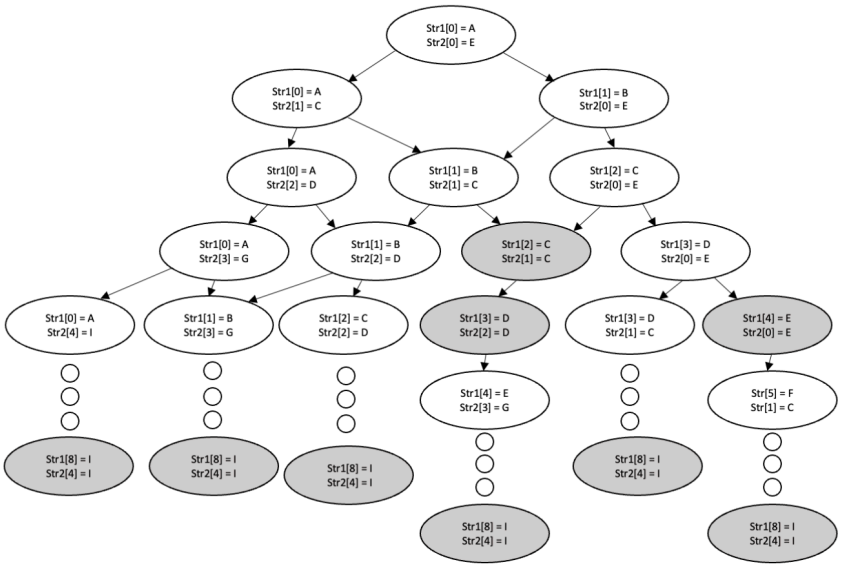
So throughout the recursion, we continuously compare the subsequences discovered from different branches and keep the longest one found so far.

Let's revisit problem (2):

$$\mathrm{S1} = \text{“ABCDEFGHI”}$$

$$\mathrm{S2} = \text{“ECDGI”}$$

At certain points, the recursion discovers valid subsequences such as:
$\text{“}CD\text{”}; \text{“}CDI\text{”}; \text{“}CGI\text{”}; \text{“}EGI\text{”}; \text{“}CDGI\text{”}$



And after exploring all possible recursive paths, the algorithm finally concludes that the longest common subsequence is: "$CDGI$"

So conceptually, Backtracking works beautifully. It guarantees that no possible subsequence is ignored. But computationally? It becomes extremely expensive.

Here's the problem. As the recursion tree grows, we begin noticing something strange: The exact same smaller problems keep appearing again and again. The algorithm repeatedly compares the same suffixes of the strings, which means same remaining characters, same recursive states, same results

In Backtracking terminology, these are called overlapping subproblems. And this is where efficiency completely collapses. Because instead of solving a subproblem once, the algorithm may solve it dozens, hundreds, or even thousands of times.

For small strings, this is manageable. But as the strings become longer, the recursion tree explodes exponentially. The algorithm spends most of its time recomputing answers it already discovered before.

And this exact weakness becomes the motivation for Dynamic Programming. Because Dynamic Programming asks a very important question:

> ***"What if we store the answers of subproblems... instead of recomputing them every single time?"***

And surprisingly, that single idea transforms the Longest Common Subsequence problem from an extremely slow recursive search into something dramatically more efficient.

3. Solution using Dynamic Programming

Now let's revisit problem (2) one last time:

$$\text{S1} = \text{"ABCDEFGHI"}$$

$$\text{S2} = \text{"ECDGI"}$$

Previously, we solved this using Backtracking. Conceptually, it worked perfectly. But computationally, it was extremely inefficient. Why? Because the same smaller subproblems kept appearing repeatedly. The algorithm continuously re-examined identical suffixes of the strings again and again, causing a huge amount of redundant computation.

And this is exactly where Dynamic Programming enters. Instead of recomputing every smaller problem repeatedly, Dynamic Programming stores the answers of smaller subproblems and reuses them whenever needed.

So rather than exploring an enormous recursive tree, we systematically build the solution step by step.

The Longest Common Subsequence problem uses the **Bottom-Up Dynamic Programming** approach. We prepare a table called: LCS → The size of this table is: Rows = length of $S1 + 1$; Columns = length of $S2 + 1$

Since: $S1 =$ "$ABCDEFGHI$" → length 9 and $S2 =$ "$ECDGI$" → length 5. The table becomes: 10 rows and 6 columns. Each position: L$CS[i][j]$ stores the length of the Longest Common Subsequence between: $S1[0 : i]$ and $S2[0 : j]$

In simpler words, every cell answers this question:

> ***"Using the first i characters of S1 and the first j characters of S2... what is the length of their LCS?"***

Step 1 — Initialize the Table

First, we initialize the entire first row and first column with 0. Why? Because if one string is empty, then no common subsequence can exist. So: $LCS[0][j] = 0$ and $LCS[i][0] = 0$ for all valid i and j. The initial table becomes:

	0	E	C	D	G	I
0	0	0	0	0	0	0
A	0					
B	0					
C	0					
D	0					
E	0					
F	0					
G	0					
H	0					
I	0					

Step 2 — Fill the Table

Now the real process begins. For every pair of characters: $S1[i]$ and $S2[j]$, two situations may occur.

- **Case 1 — Characters Match**

If the characters are equal: *Extend the previous subsequence.* So we take:

$$LCS[i][j] = 1 + LCS[i-1][j-1]$$

The diagonal position represents the subsequence length before this new matching character was added.

- **Case 2 — Characters Do Not Match**

If the characters differ, we carry forward the best subsequence found so far. So we take:

$$LCS[i][j] = \max(LCS[i-1][j],\ LCS[i][j-1])$$

This means: Either ignore the current character from $S1$ or ignore the current character from $S2$**.** And choose whichever gives the larger subsequence.

Filling the First Few Cells

We start at: $LCS[1][1]$. This compares: "A" and "E". They do not match. So:

$$LCS[1][1] = \max(LCS[0][1], LCS[1][0]) = 0$$

The cell becomes 0. The same process continues until we reach: $LCS[3][2]$. This compares: "C"from $S1$ and "C" from $S2$. Now the characters match. So:

$$LCS[3][2] = 1 + LCS[2][1] = 1$$

The value becomes 1.

Next: $LCS[4][3]$ compares: "D" from $S1$ and "D" from $S2$. Another match. So:

$$LCS[4][3] = 1 + LCS[3][2] = 2$$

And the process continues until the entire table is filled.

Final DP Table

	0	E	C	D	G	I
0	0	0	0	0	0	0
A	0	0	0	0	0	0
B	0	0	0	0	0	0
C	0	0	1	1	1	1
D	0	0	1	2	2	2
E	0	1	1	2	2	2
F	0	1	1	2	2	2
G	0	1	1	2	3	3
H	0	1	1	2	3	3
I	0	1	1	2	3	4

The final value is found at: $LCS[9][5] = 4$. Which means: The longest common subsequence has length 4. But we still do not know the subsequence itself. So now we backtrack.

Step 3 — Backtracking to Construct the LCS

Starting from the bottom-right corner: $LCS[9][5] = 4$. We compare neighboring cells: Up → $LCS[8][5]$, Left → $LCS[9][4]$

If the current value exists in one of those positions, we move there. But if the value does **not** exist in either neighboring position, then the current character pair must have matched. And whenever that happens, we move diagonally, and take that character as part of the LCS

Example of the Backtracking Process

- Starting at: $LCS[9][5] = 4$. The value 4 does not exist in: $LCS[8][5]$ and $LCS[9][4]$. So this means: "I" from $S1$ matched with "I" from $S2$

⇒ We move diagonally to: $LCS[8][4]$ and record: "I"

- Next: $LCS[8][4] = 3$. The value 3 exists at: $LCS[7][4]$. So we move upward at: $LCS[7][4] = 3$. The value 3 does not exist in: $LCS[6][4]$ and $LCS[7][3]$. So another diagonal occurs. This gives: "G"
- And the process repeats until we eventually reconstruct: "C", "D", "G", "I"

Thus, the Longest Common Subsequence becomes:

$$\text{“}CDGI\text{”}$$

Complexity Analysis

Let: $m =$ length of $S1$ and $n =$ length of $S2$. The algorithm fills an entire table of size: $m \times n$. So:

- Time Complexity: $O(m \times n)$
- Space Complexity: $O(m \times n)$ because the DP table must be stored.

However, space optimization techniques such as rolling arrays can reduce the space complexity to: $O(\min(m, n))$

Below is the simple Python implementation of LCS using Dynamic Programming:

```python
import numpy as np


def lcs(s1, s2, m, n):
    dp = np.zeros((m + 1, n + 1), int)
    for i in range(m + 1):
        for j in range(n + 1):
            if i == 0 or j == 0:
                dp[i][j] = 0
            elif s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    longest_com = ""

    i = m
    j = n
    while i > 0 and j > 0:
        # If S1 and S2 is the same then it is LCS
        if s1[i - 1] == s2[j - 1]:
            longest_com += s1[i - 1]
            i -= 1
            j -= 1
        # If not, find the better value
        elif dp[i - 1][j] > dp[i][j - 1]:
            i -= 1
        else:
            j -= 1

    # Reverse to get the correct LCS
    longest_com = longest_com[::-1]
    print("Longest Common Subsequence:", longest_com)
```

The function requites the two strings s1 and s2, alongside its length m and n respectively

```
s1 = "ABCDEFGHI"
s2 = "ECDGI"
m, n = len(s1), len(s2)
lcs(s1, s2, m, n)
```

Longest Common Subsequence: CDGI

Summary

Compared to Backtracking, Dynamic Programming is dramatically more efficient. Backtracking repeatedly recomputes identical subproblems. Dynamic Programming solves each subproblem only once.

And this is the true power of DP: ***It transforms repeated recursive exploration into structured memory-based computation.***

Instead of blindly exploring every possibility, Dynamic Programming systematically builds knowledge from smaller solved problems.

Which is why the Longest Common Subsequence problem is considered one of the classic demonstrations of Dynamic Programming itself.

References

https://www.geeksforgeeks.org/dsa/longest-common-subsequence-dp-4/

https://www.enjoyalgorithms.com/blog/longest-common-subsequence

https://www.youtube.com/watch?v=sSno9rV8Rhg

https://www.youtube.com/watch?v=4ClOkX0SWW4